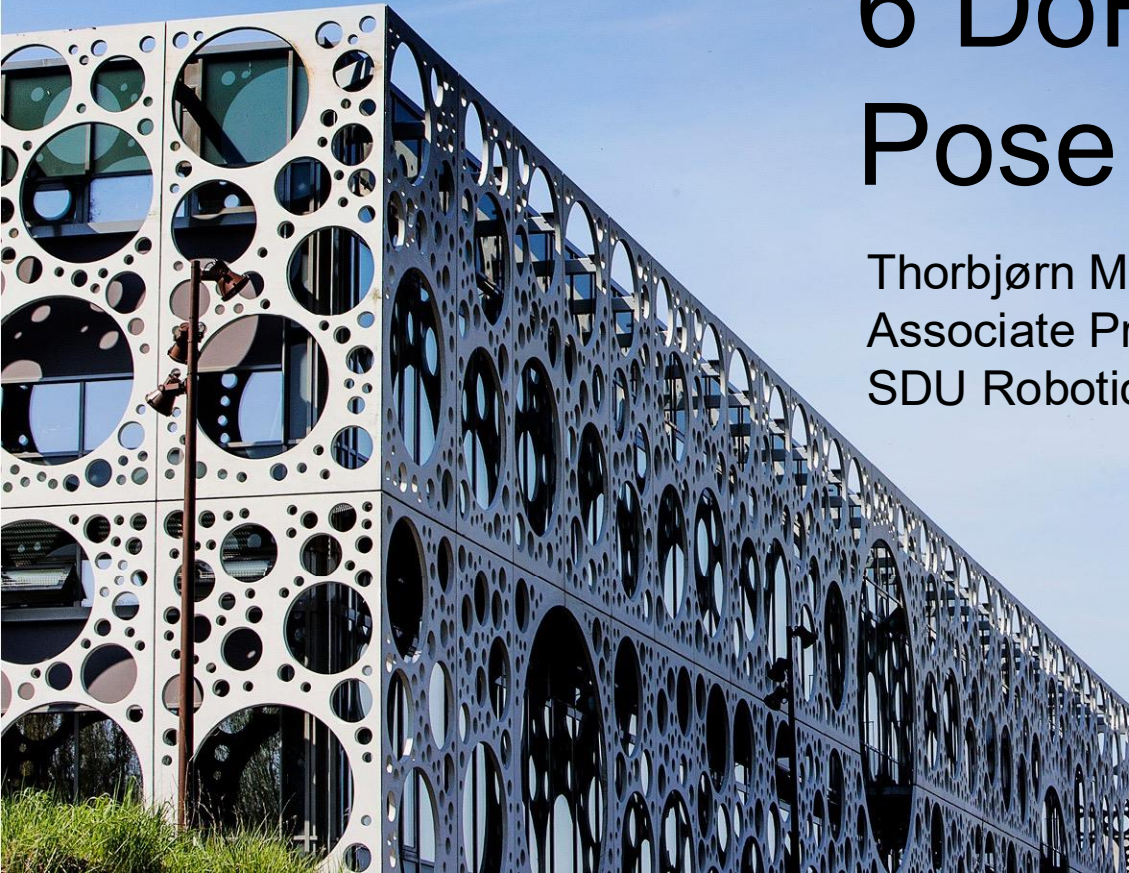


# 6 DoF Object Pose Estimation

Thorbjørn Mosekjær Iversen  
Associate Professor  
SDU Robotics



# About me

## **Thorbjørn Mosekjær Iversen**

Associate Professor  
SDU Robotics  
The Maersk Mc-Kinney Moller Institute  
University of Southern Denmark

Research interests include

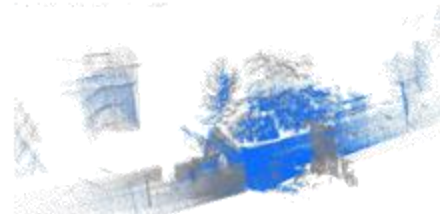
- Industrial computer vision
- Uncertainty estimation

Contact: [thmi@mmmi.sdu.dk](mailto:thmi@mmmi.sdu.dk)



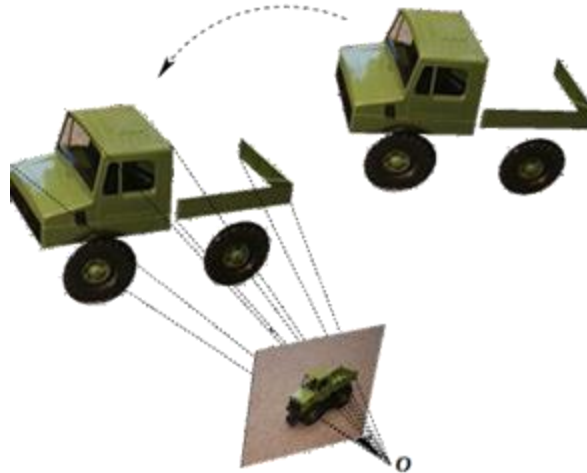
# Why pose estimation

- Robotic automation
- Grasping
- Registration
- Augmented reality



# Pose

- One definition:
  - ... *determining a camera's position relative to a known 3D object or scene...*
- Another definition
  - ... *the transformation needed to map an object model from its inherent coordinate system into agreement with the sensory data.*



# Pose

- Rotates and translates the object (points) into a scene

$$q = Rp + t = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p \\ 1 \end{bmatrix} \quad T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$$

- $R$ : rotation

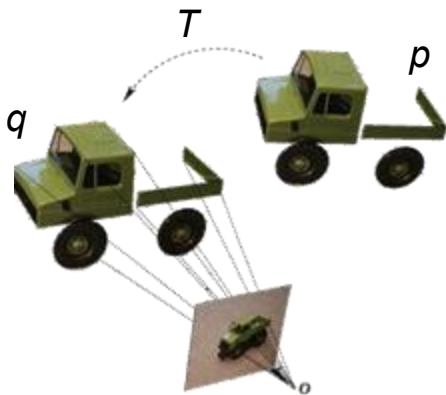
3x3 matrix in  $SO(3)$

$$\begin{bmatrix} R_{00} & R_{01} & R_{02} \\ R_{10} & R_{11} & R_{12} \\ R_{20} & R_{21} & R_{22} \end{bmatrix}$$

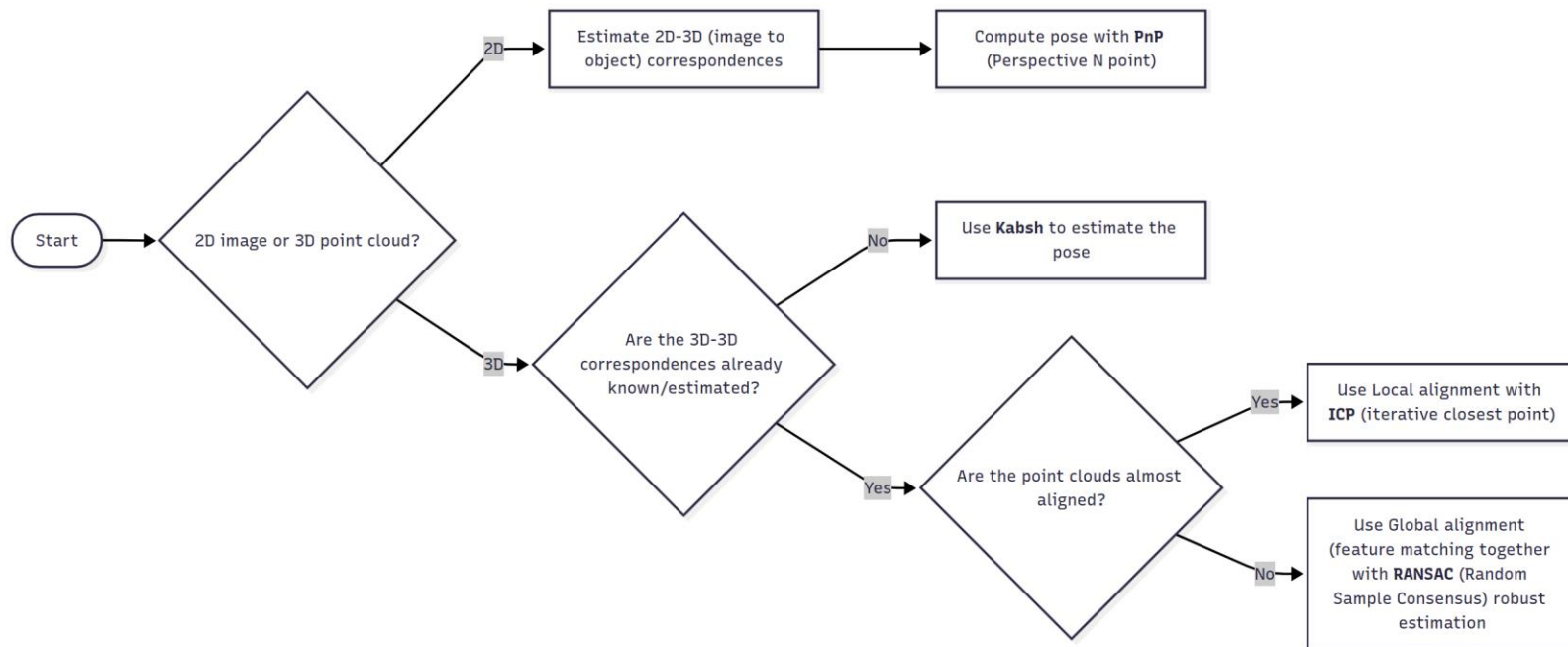
- $t$ : translation

3x1 vector in  $R^3$

$$\begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}$$



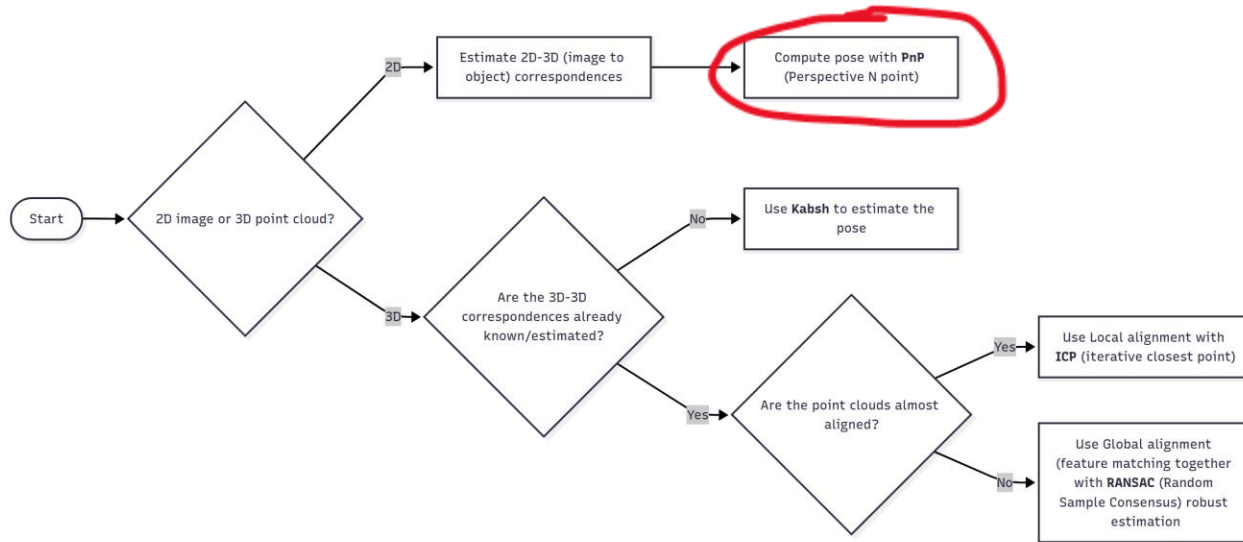
# Reference flow chart



# Transformation estimation

- Objective
  - Given **matching** sensor/object points, find the pose
- Two scenarios
  - 3D-2D
    - Object-relative 3D points matched with image points
  - 3D-3D
    - Object-relative 3D points matched with 3D points







# 3D-2D transformation estimation

- Each object point  $p$  undergoes a projection

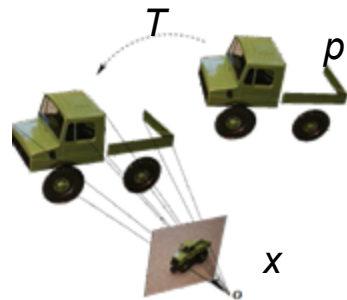
$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & \gamma & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

2D homogenous  
image point  $\mathbf{x}$

Camera  
matrix  $\mathbf{K}$

Pose  
 $\mathbf{R} \mathbf{t}$

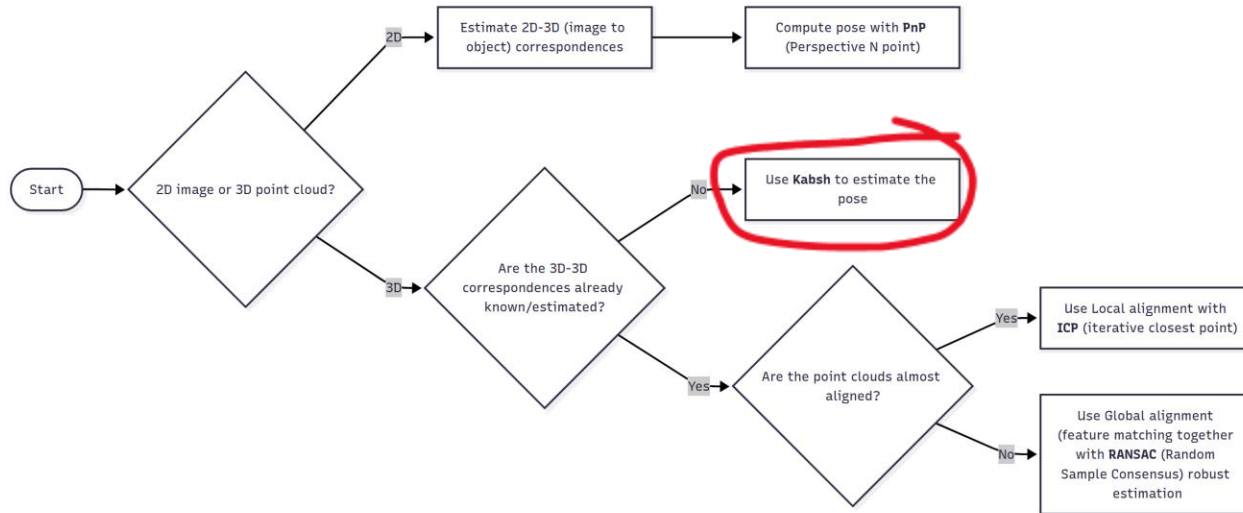
3D homogeneous point  
in object frame  $\mathbf{p}$



- Based on *matching*  $(x,p)$  pairs, find  $\mathbf{R}$  and  $\mathbf{t}$  ( $\mathbf{K}$  is known by calibration)
  - This is the **perspective-n-point** problem (PnP)

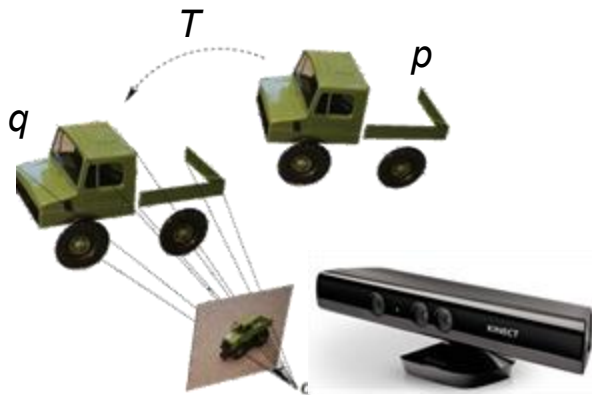
Note: with 3 points (p3p) we can get up to 4 analytical solutions, so we usually use a minimum of four points

Gao, Xiao-Shan, et al. "Complete solution classification for the perspective-three-point problem." *IEEE transactions on pattern analysis and machine intelligence* 25.8 (2003): 930-943.



# 3D-3D transformation estimation

- If we reconstruct the object, e.g. using stereo, we can directly observe the 3D scene points  $q$ 
  - $q = Tp = [R \mid t]p$
- Based on *matching*  $(p, q)$  pairs, find can compute  $R$  and  $t$  using **Kabsh** algorithm



# 3D-3D transformation estimation

- The Kabsch algorithm

- Compute centroids:

$$\mathbf{c}_p = \frac{1}{n} \sum_{i=1}^n \mathbf{p}_i, \quad \mathbf{c}_q = \frac{1}{n} \sum_{i=1}^n \mathbf{q}_i$$

- Demean the point sets:

$$\mathbf{p}'_i = \mathbf{p}_i - \mathbf{c}_p, \quad \mathbf{q}'_i = \mathbf{q}_i - \mathbf{c}_q$$

- Compute cross cov.:

$$\mathbf{C}_{pq} = \sum_{i=1}^n \mathbf{p}'_i (\mathbf{q}'_i)^\top$$

- Take SVD:

$$\mathbf{C}_{pq} = \mathbf{U} \mathbf{S} \mathbf{V}^\top$$

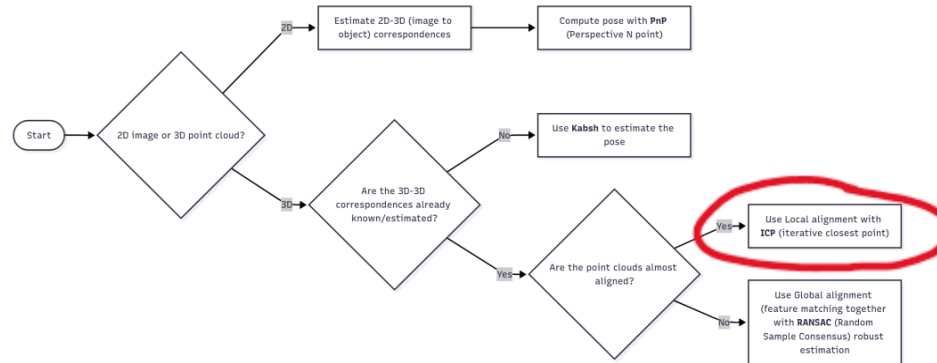
- Rotation and translation:

$$\begin{aligned} \mathbf{R} &= \mathbf{V} \mathbf{S}' \mathbf{U}^\top & \mathbf{S}' &= \text{diag}(1, 1, \dots, \det(\mathbf{V} \mathbf{U}^\top)) \\ \mathbf{t} &= \mathbf{c}_q - \mathbf{R} \mathbf{c}_p \end{aligned}$$

# But wait!!! How do we know which object points corresponds to which scene points?!?

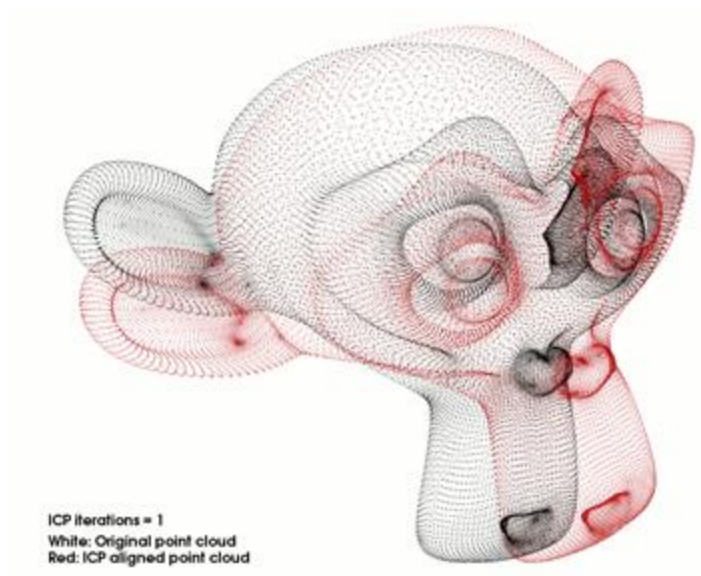
- We don't!
- This means we have to find ways to match points in the object's frame to points in the scene
  - RGB cameras: match 2D image points
  - Depth or stereo: match 3D scene points
- This is the real challenge in Pose estimation

# Local alignment



# Local alignment

- Definition
  - Given two 3D models (object and scene) that are *almost aligned*, find the pose that brings them into optimal alignment





# Iterative Closest Point

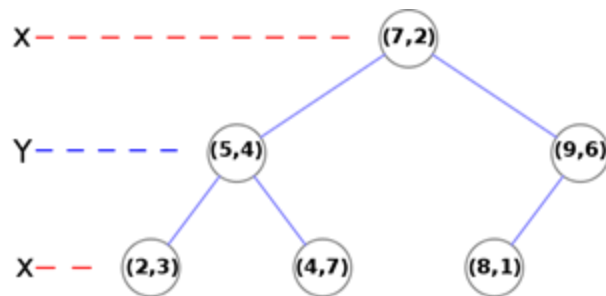
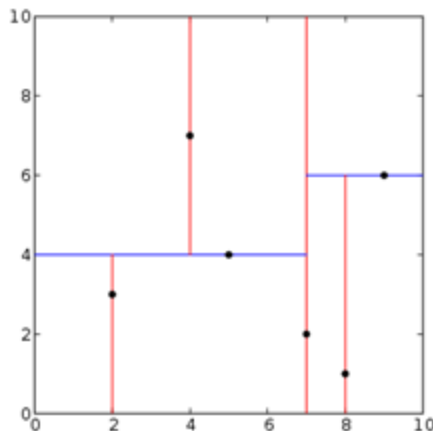
- The ICP algorithm for local alignment
  - 1) For each object point  $p$ , find the *nearest* scene point  $q$
  - 2) Use all  $(p, q)$  pairs to estimate  $T$  using the Kabsch algorithm
  - 3) Apply  $T$  to the object points  $p$  and go to 1)



# Iterative Closest Point

1) For each object point  $p$ , find the *nearest* scene point  $q$

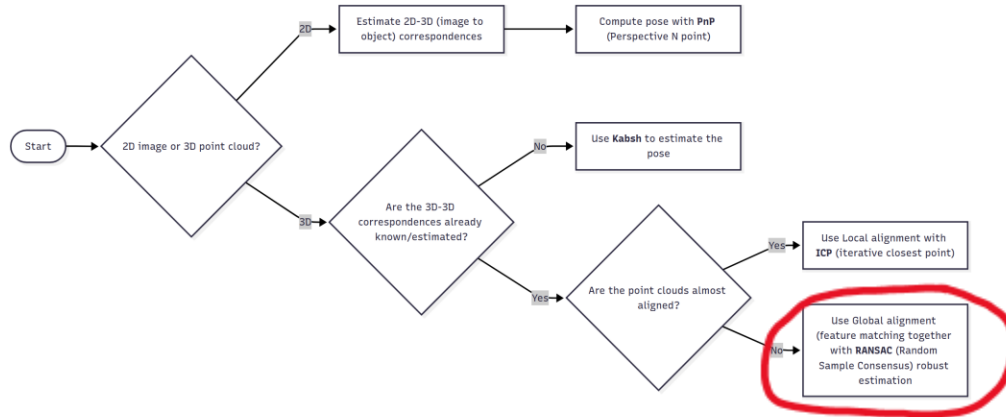
- If there are  $N$  object points and  $M$  scene points, this requires  $NM$  distance computations
- Luckily  $k$ -d trees reduce this complexity to  $N \cdot \log(M)$



# Iterative Closest Point

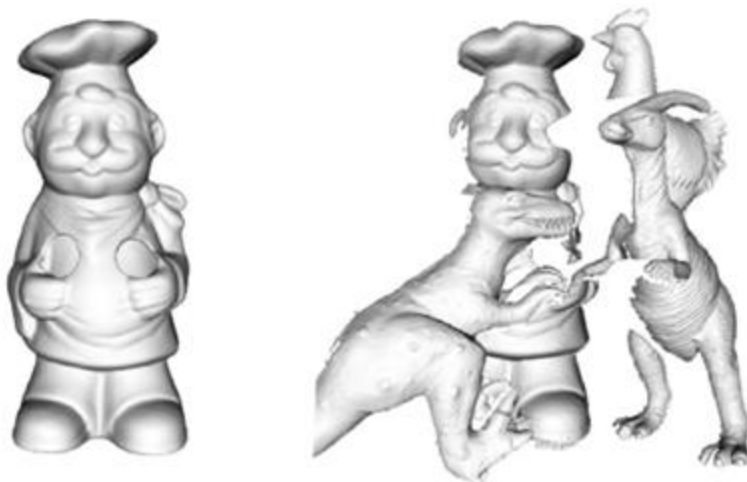
- When does the algorithm stop?
  - After a fixed number of iterations
  - When the difference between two consecutive transformations is below some threshold
  - When the mean alignment error is below some threshold  $E[ (p - q)^T(p - q) ] < \varepsilon$

# Global alignment



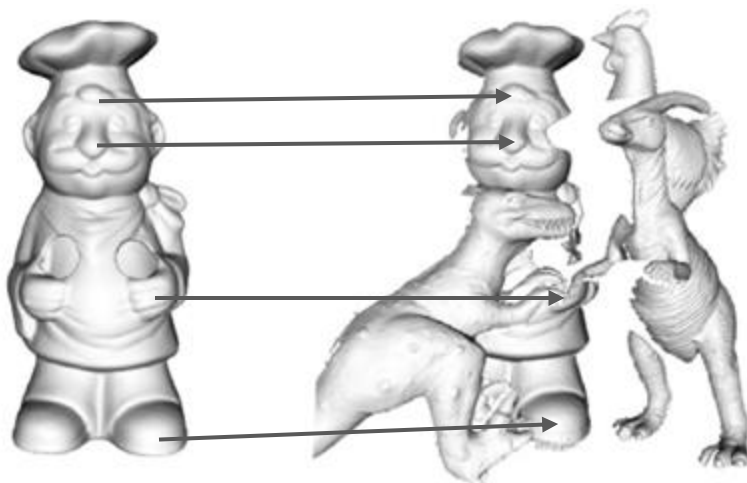
# Global alignment

- A more general case
  - Given two 3D models *in any relative position*, find the pose that brings them into optimal alignment
- Also termed free-form pose estimation



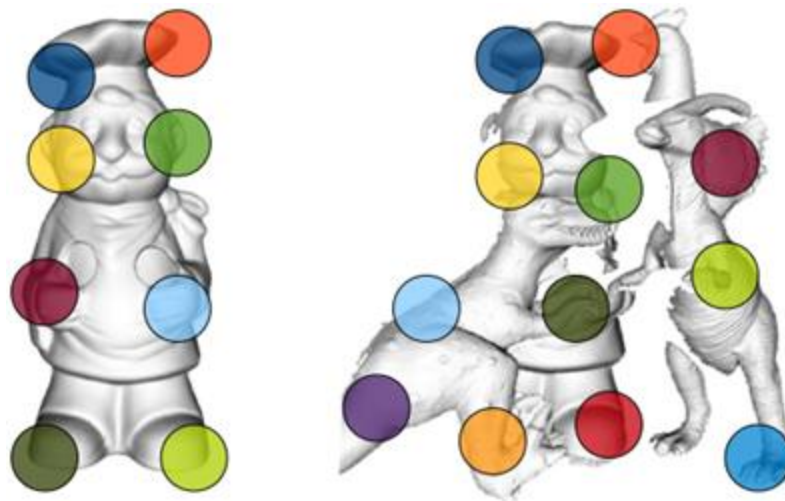
# The correspondence problem

- Local alignment (ICP) will fail
  - Closest point assignments are invalid
- The correspondence problem
  - How do we associate points that are not close?



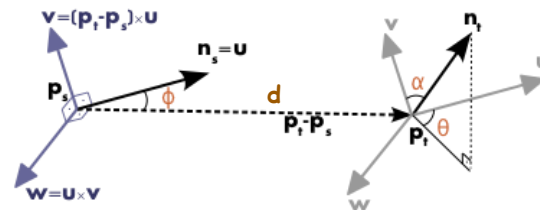
# Local shape features

- Describe each surface point in an invariant manner
- Each shape feature describes the point using some statistics
- Support radius  $r$  determines the scale of the feature
- Correspondences: matched object features

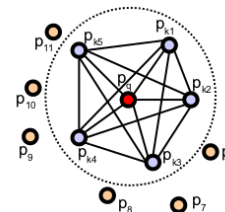


# Local shape features

- Point feature histogram (PFH)
  - Find all oriented points in a spherical neighborhood of radius  $r$  around each point ( $k$ -d tree!)
  - Compute relative angles using surface normals and direction vector from the source point to each neighbor
  - Bin the data in a 4D-histogram
- Real time variants are often used, e.g. Fast Point Feature Histograms (FPFH)



[https://pcl.readthedocs.io/projects/tutorials/en/latest/pfh\\_estimation.html#pfh-estimation](https://pcl.readthedocs.io/projects/tutorials/en/latest/pfh_estimation.html#pfh-estimation)

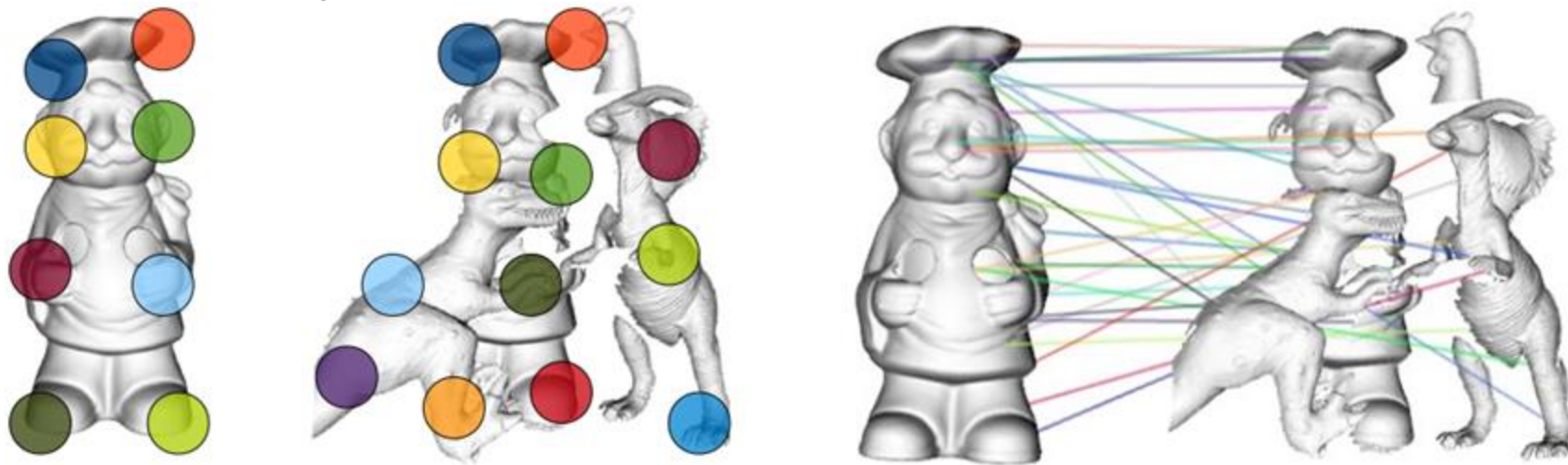


R. B. et al.: Fast Point Feature Histograms (FPFH) for 3D Registration. ICRA, 2009.



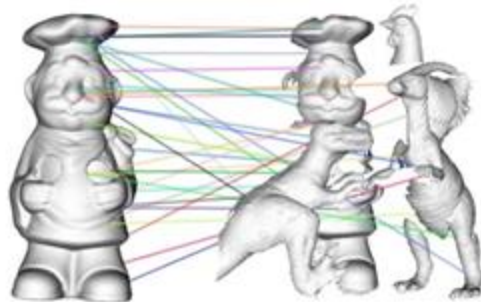
# Feature matching

- Matching: find the  $k$  nearest neighbors by a search in high-dimensional feature spaces (often  $k = 1$ )
- These are pose-invariant point associations
- Expect many *outliers* and few *inliers*



# Robust pose estimation

- Inputs
  - Object model (point cloud with normals)
  - Scene model (point cloud with normals)
  - Feature matches (badly) contaminated by outliers
- Objective
  - Find the relative pose by sampling subsets of the correspondences



# RANSAC

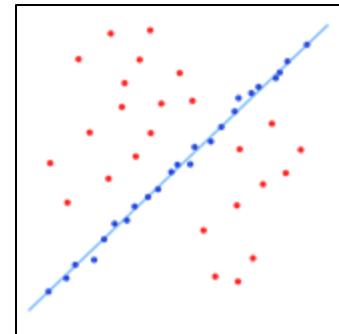
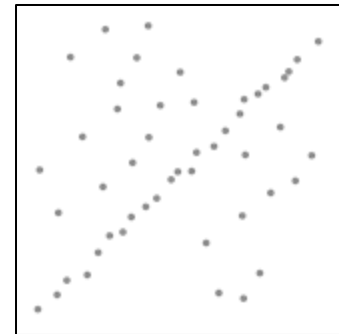
- A general algorithm for fitting models to data
- Line example - repeat the following:

1) Sample 2 random points

2) Fit a line to the points

3) Count how many other points (inliers) lie close to the line

4) If inlier count is the current maximum, save model



# RANSAC pose estimation

- In this case the “model” is the pose
- The “data” is the set of matches, which we sample

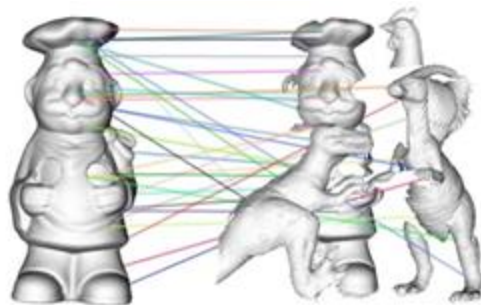
1) Sample 3 point pairs from feature matches

2) Estimate relative pose (Kabsch)

3) Apply pose to object and *validate* the pose

- e.g. counting number of inliers

4) If pose is best so far, save it



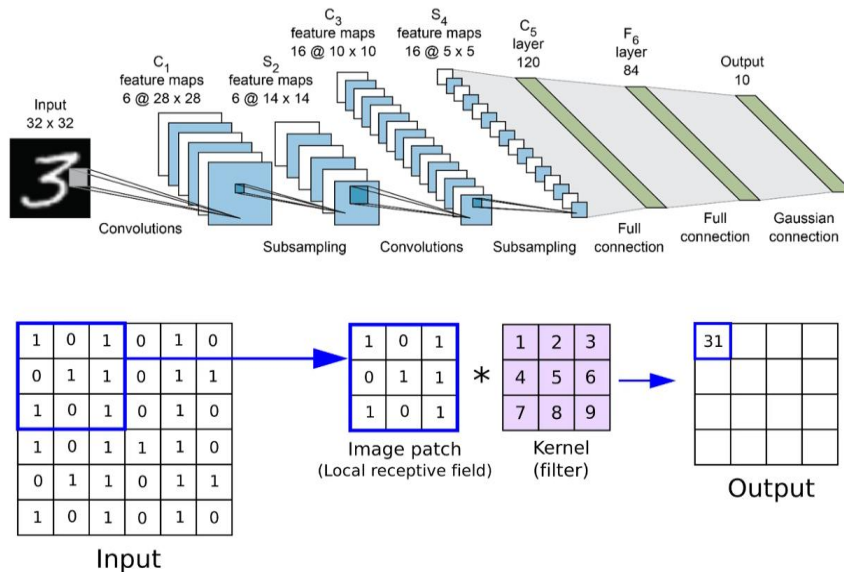
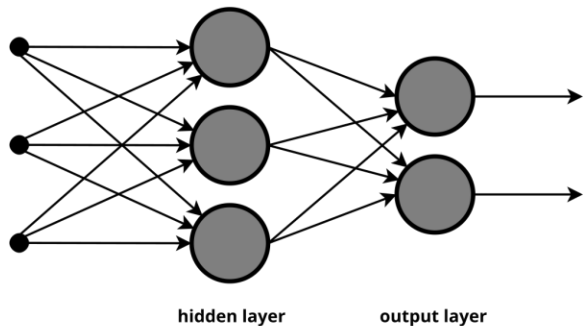
# Classical methods and deep learning

# Overview

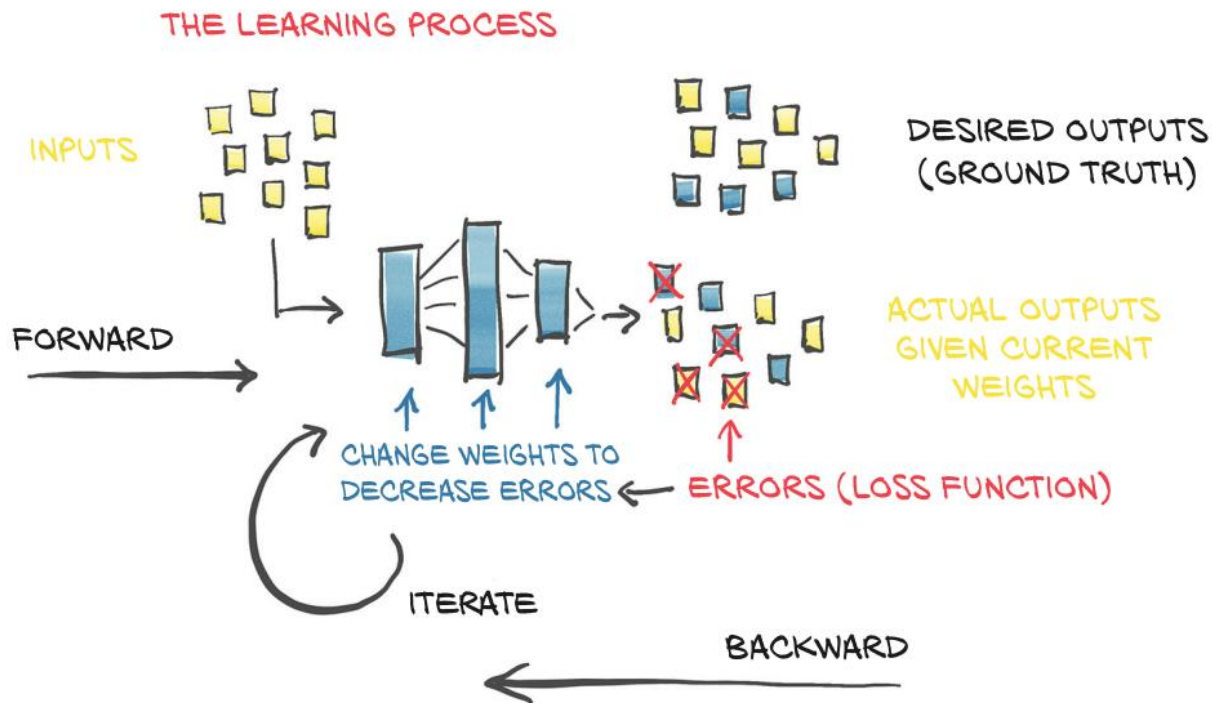
- Convolutional neural networks (CNNs)
- Deep learning based pose estimation from single RGB images (SurfEmb)
  - Convolutional neural networks
  - Object detection (YOLO)
- Extension to multiview (Multiview SurfEmb)
- The future of pose estimation (SpyroPose)

# Convolutional neural networks (CNNs)

- The convolutional kernel contains the networks weights
- A purely convolutional network has translational equivance



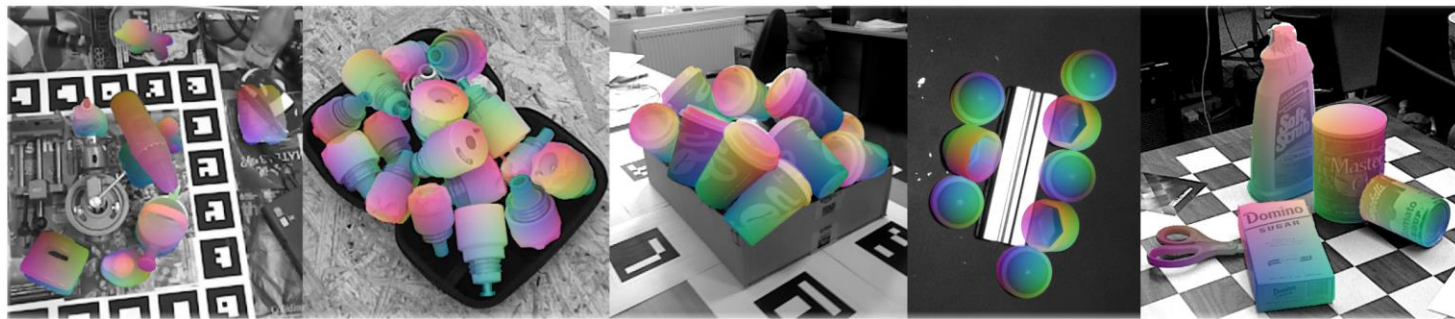
# Supervised training of a neural network





# SurfEmb

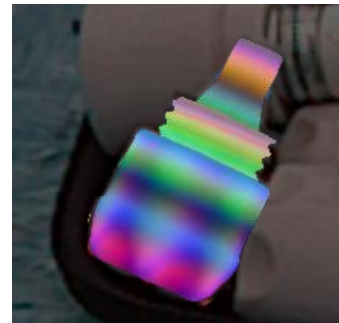
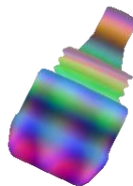
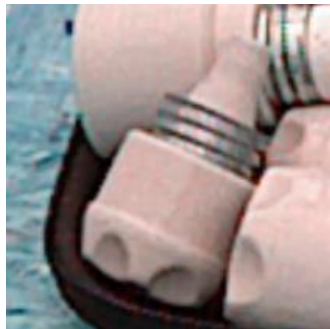
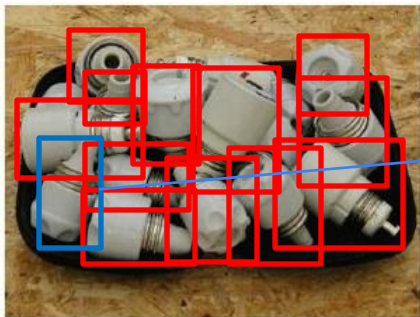
- 6D Object Pose estimation from a single RGB image
- Trained purely in simulation
- State of the art when published in 2022
- An example of how "ancient" concepts such as PnP and Ransac are still used in modern deep learning based pose estimation methods



# SurfEmb

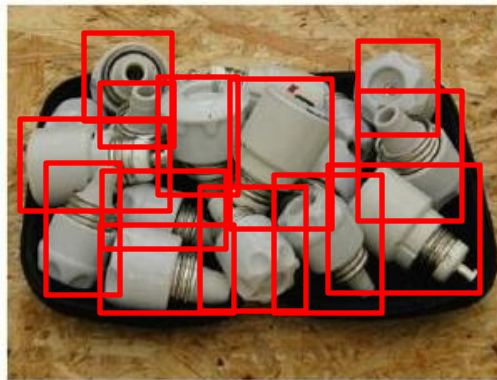
Method:

1. An object detector to find ROIs of individual object instances
2. Deep learning to find 3D-2D correspondences (implicitly learns symmetries)
3. PnP to provide pose hypotheses from correspondences
4. Ransac to handle the many outlier hypotheses
5. Local refinement using rendering



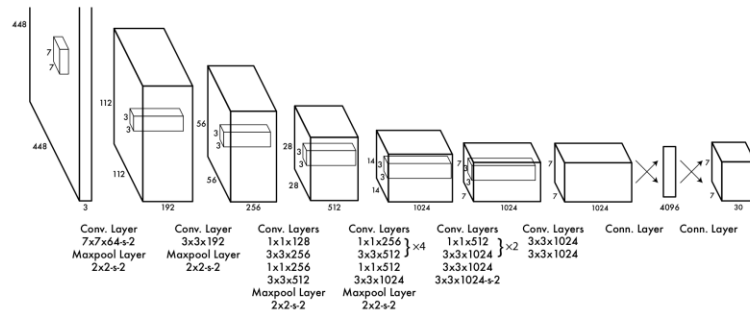
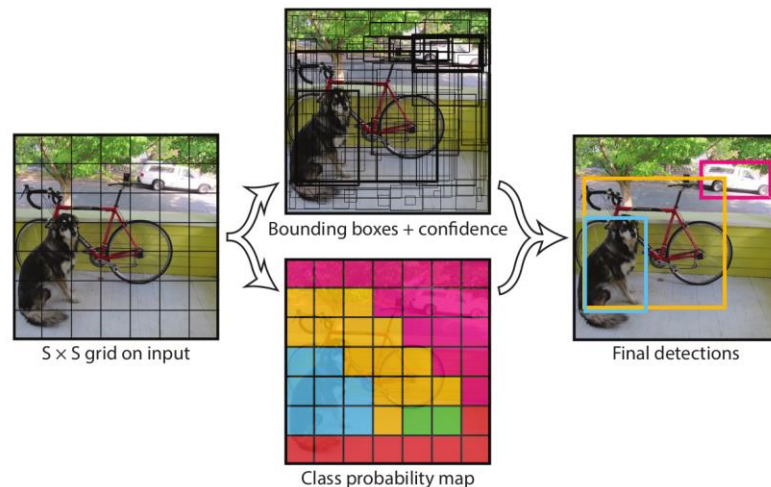
# Object detection

- Find out what objects are in the image and where they are
- Often used as initial step in pose estimation from 2D, to provide ROIs of individual object instances



# YOLO

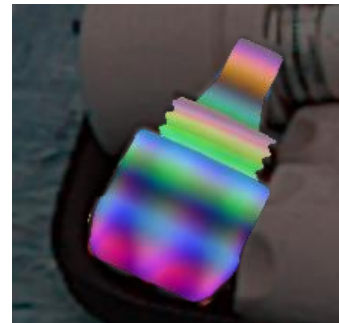
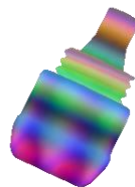
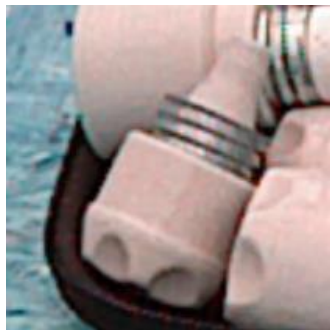
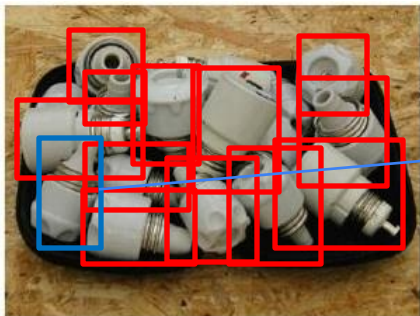
- End-to-end real time object detection
- For each region in a  $S \times S$  grid estimates
  - $B$  bounding boxes, each with parameters
    - Bounding box center ( $x, y$ )
    - Bounding box size ( $h, w$ )
    - Confidence that an object is present
  - $C$  class probabilities



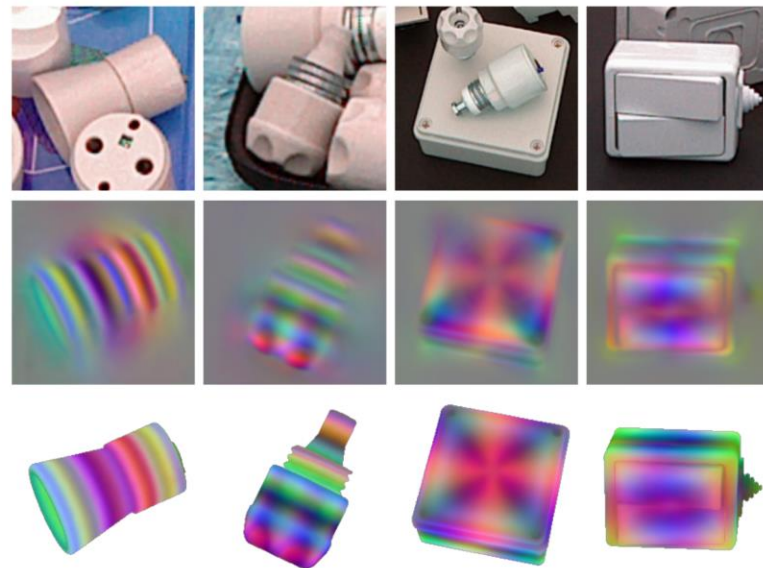
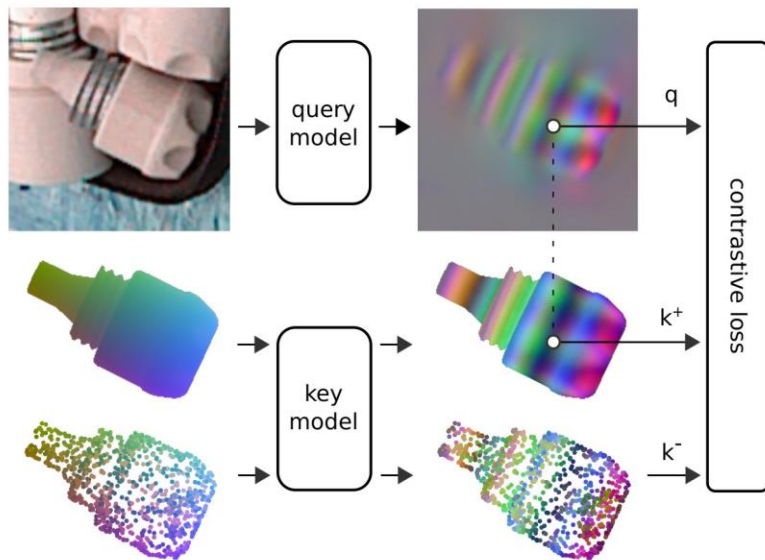
# SurfEmb

Method:

1. An object detector to find ROIs of individual object instances
2. Deep learning to find 3D-2D correspondences (implicitly learns symmetries)
3. PnP to provide pose hypotheses from correspondences
4. Ransac to handle the many outlier hypotheses
5. Local refinement using rendering



# Contrastive learning for finding 3D-2D correspondences

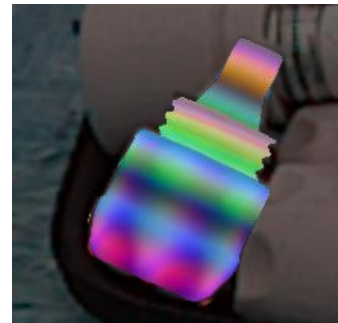
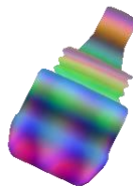
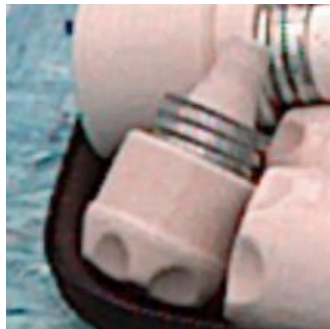
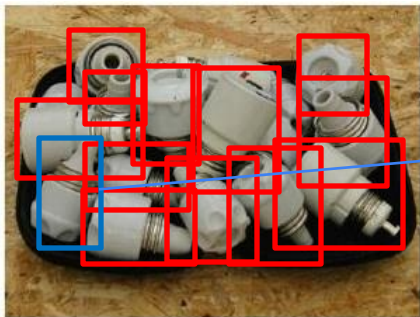




# SurfEmb

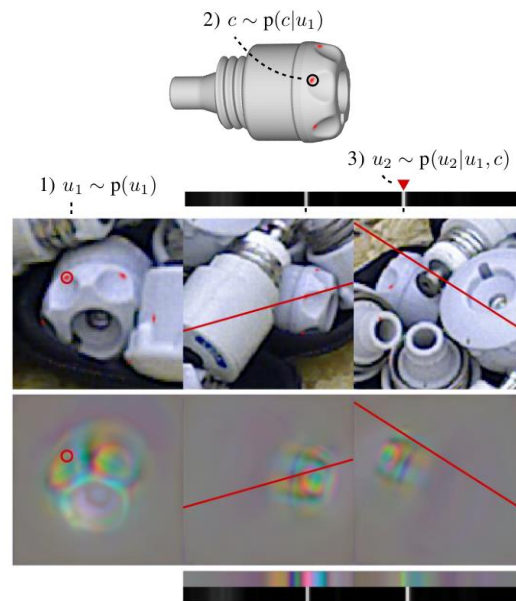
Uses:

1. An object detector to find ROIs of individual object instances
2. Deep learning to find 3D-2D correspondences (implicitly learns symmetries)
3. PnP to provide pose hypotheses from correspondences
4. Ransac to handle the many outlier hypotheses
5. Local refinement using rendering



# Multiview SurfEmb

- Surface embeddings for calibrated multiview triangulation
  - Find matching model and scene point pairs  $(c, x)$ :
    - Sample point in one image ( $u_1$ )
    - Sample point on 3D object using learned surface embeddings ( $c$ )
    - Sample point in one of the other images using epipolar lines and surface embeddings ( $u_2$ )
    - Triangulate 3D point ( $x$ )
  - Use Kabsh to compute pose hypotheses
  - Check geometric consistency to remove invalid matches





# SpyroPose

- Pose distribution estimation
  - Estimate probability distributions over object poses, instead of point estimates
- Implicit modeling of the pdf
  - Multimodal parameter free distributions
- Uses hierarchical pose pyramids
  - Only expanded high density regions in 6D pose space

